

Dossier de Validation Technique

Plateforme d'Automatisation Intelligente des Tests Applicatifs

Document de préparation à la validation technique (soutenance PFE). Auteur : Ilyes Chaabani — Version 1.0

Ce dossier contient : (1) la vue d'ensemble et le pitch, (2) l'architecture technique détaillée, (3) le deep-dive de chaque module, (4) un scénario de démo chronométré et reproductible, (5) la justification des décisions techniques, (6) la préparation aux questions du jury, (7) les limites connues et leurs réponses honnêtes, (8) la checklist pré-démo.

1. Vue d'ensemble (le pitch — 2 minutes)

1.1 Le problème

Dans une équipe de développement bancaire, **écrire et maintenir les tests** (unitaires, intégration, end-to-end, sécurité) coûte cher en temps humain. Les tests sont : - longs à écrire manuellement, - vite obsolètes quand le code change, - inégalement couverts (la sécurité et l'UX sont souvent négligées), - difficiles à piloter (qui exécute quoi, qui corrige quel échec ?).

1.2 La solution

Une **plateforme web unifiée** qui automatise **tout le cycle de vie du test** grâce à l'IA :

1. **Génération automatique** de tests par un LLM (à partir du code source réel du projet).
2. **Exécution réelle** des tests dans un environnement isolé (clone Git + Maven).
3. **Auto-correction** : si un test généré échoue à la compilation/exécution, l'IA le corrige seule (jusqu'à 3 tentatives).
4. **Agent autonome d'évaluation UX/fonctionnelle** qui navigue l'application comme un humain (via un LLM de vision) et teste les formulaires.
5. **Scans de sécurité réels** : SAST (Semgrep), DAST (OWASP ZAP), SCA (OWASP Dependency-Check) avec mapping OWASP Top 10.
6. **Rapports PDF** professionnels générés automatiquement + analyse IA des échecs.
7. **Pilotage collaboratif** : projets, rôles, assignments, notifications.

1.3 La phrase qui résume

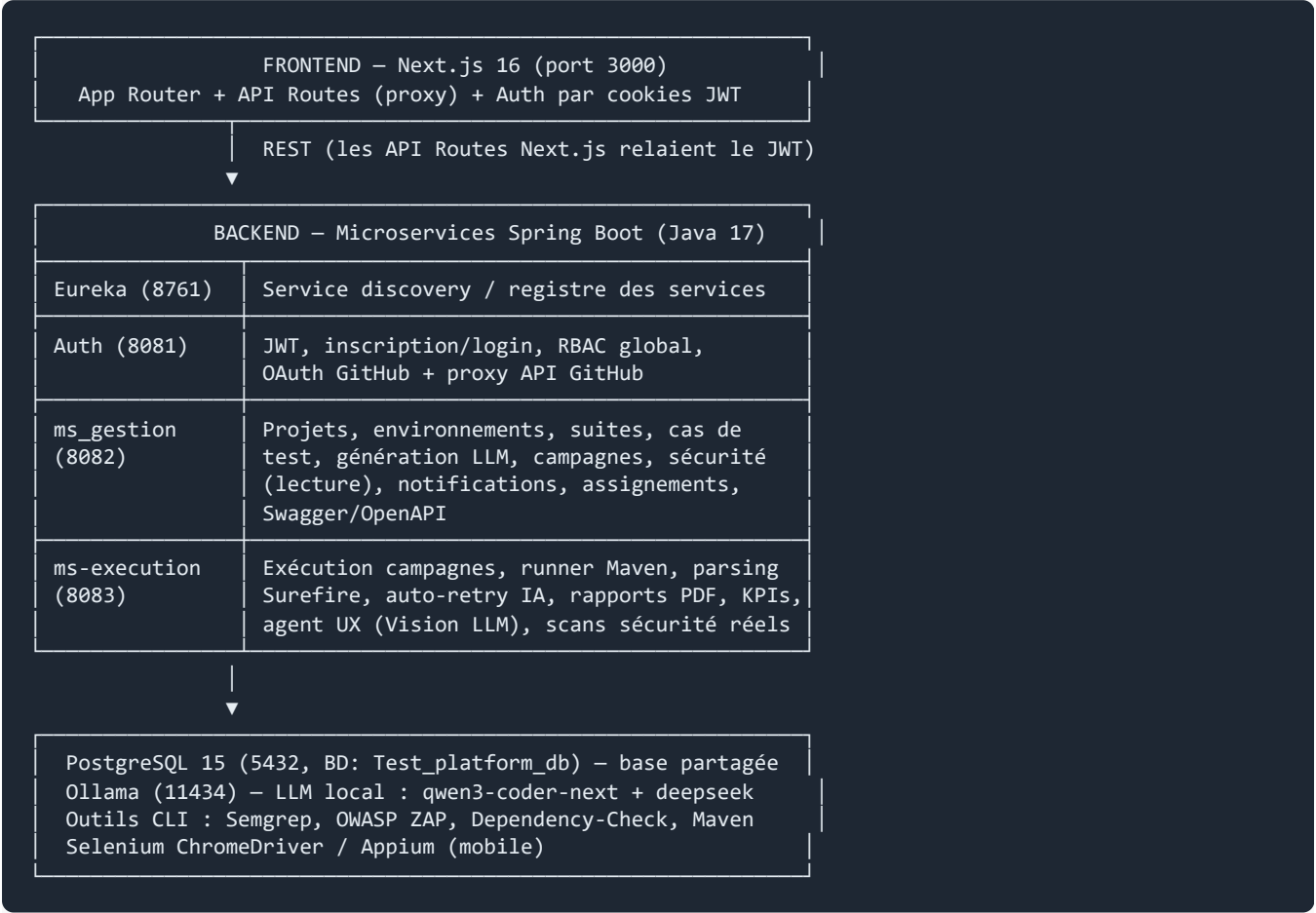
« C'est une plateforme qui transforme le code source d'une application en tests exécutables, les lance réellement, se corrige toute seule en cas d'échec, et y ajoute une couche d'audit sécurité et UX pilotée par IA — le tout dans une interface collaborative. »

1.4 Ce qui la distingue (les arguments forts)

- **L'IA ne fait pas que suggérer du texte** : elle génère du code qui est *réellement compilé et exécuté*, puis *auto-corrigé* en boucle fermée. C'est de l'IA *agentique*, pas un simple chatbot.
 - **Vision LLM** : l'agent UX « voit » réellement les captures d'écran de l'application et décide de l'action suivante.
 - **Sécurité réelle** : ce ne sont pas des résultats simulés — on exécute de vrais outils standards de l'industrie (Semgrep, ZAP, Dependency-Check).
 - **Architecture microservices** propre, avec service discovery (Eureka) et séparation des responsabilités.
-

2. Architecture technique

2.1 Schéma global



2.2 Stack technique

Couche	Technologie	Justification
Frontend	Next.js 16 (App Router), React, TypeScript, Tailwind	SSR + API routes intégrées = un seul déploiement front qui sert aussi de proxy sécurisé
Backend	Spring Boot 3, Java 17	Standard entreprise, écosystème mature, Spring Security
Service discovery	Netflix Eureka	Découplage des services, pas d'URLs en dur entre microservices
Base de données	PostgreSQL 15	Robuste, support JSONB (utilisé pour <code>test_data</code>)
ORM	Hibernate / Spring Data JPA	Productivité, mapping objet-relationnel
LLM génération	Ollama — <code>qwen3-coder-next:cloud</code> (primaire), <code>deepseek-coder:6.7b</code> (fallback)	LLM local = pas de fuite de code source vers un tiers, coût nul
LLM vision	OpenRouter Vision ou Ollama Vision (provider commutable)	L'agent UX a besoin de « voir » les captures d'écran
Auth	JWT (access + refresh) en cookies HttpOnly	Standard, stateless
Tests générés	TestNG + Mockito + Selenium + REST Assured + Appium + Testcontainers	Frameworks standards selon le type de test
Sécurité SAST	Semgrep	Outil open-source de référence, multi-langage
Sécurité DAST	OWASP ZAP	Standard industrie pour le scan dynamique
Sécurité SCA	OWASP Dependency-Check	Détection des CVE dans les dépendances
Git	JGit (clone programmatique)	Clone des repos à tester sans dépendre d'un Git CLI
Rapports	iTextPDF	Génération PDF programmatique

2.3 La base de données partagée (point d'architecture important)

Tous les microservices partagent **une seule base PostgreSQL**. Conséquence assumée : - **ms_gestion** est le *propriétaire du schéma* : il utilise `ddl-auto=update` (Hibernate crée/modifie les colonnes automatiquement). - **ms-execution** lit/écrit dans les mêmes tables via des **entités miroirs en lecture** (ex. `ExecutionResultRef`, `ProjectRepository`), en `ddl-auto=validate` (ne modifie jamais le schéma).

⚠ **À savoir pour le jury** : c'est une *base partagée* (shared database), pas une *database-per-service* pure. C'est un choix pragmatique pour un PFE (simplicité, cohérence transactionnelle immédiate). Voir §7 pour la réponse honnête sur ce compromis.

2.4 Flux d'authentification

1. L'utilisateur se connecte → Auth (8081) émet un **access_token** et un **refresh_token** stockés en cookies.
2. Le Frontend appelle ses propres **API Routes Next.js** (`/api/...`).
3. Chaque API Route **relaie le JWT** vers le backend via l'utilitaire `buildMsGestionHeaders()` qui transforme le cookie `access_token` en header `Authorization: Bearer ...`.
4. `ms_gestion` applique Spring Security (`.anyRequest().authenticated()`), sauf les endpoints `/api/internal/**` (service-to-service, sans contexte utilisateur).

💡 C'est exactement le bug que j'ai corrigé récemment : certaines API Routes n'appelaient pas `buildMsGestionHeaders()`, donc le JWT n'était pas transmis → 401 et page « Mes Assignements » vide. La correction a consisté à appliquer ce relais uniformément.

3. Deep-dive des modules

3.1 Authentification & RBAC (Auth — 8081)

- **Inscription / connexion** classiques (mot de passe encodé BCrypt).
- **JWT** : access + refresh token, secret partagé entre Auth et `ms_gestion` (`jwt.secret`).
- **Rôles globaux** (cumulables, principe de « badges ») : `ADMIN`, `TEST_MANAGER`, `QA_ENGINEER`, `DEVELOPER`, `VIEWER`, `TESTER`.
- Seul `ADMIN` gère les rôles des autres (`UserAdminController`).
- Certaines actions sont protégées par `@PreAuthorize("hasAnyRole('ADMIN','TEST_MANAGER')")` (ex. assigner/changer le statut d'une vulnérabilité).
- **Intégration GitHub** : OAuth + proxy de l'API GitHub (`GitHubClient` avec WebClient). Permet de **parcourir les fichiers source** d'un repo sans le cloner (pour le wizard de génération).
- **AdminBootstrap** : crée un compte admin au démarrage si absent.

3.2 Gestion : projets, environnements, suites, cas de test (ms_gestion — 8082)

Hiérarchie de configuration (toute la config technique est portée par l'**Environnement**, pas par le Projet) :

Project	→ nom + description (+ createdBy = propriétaire)
Environment	→ baseUrlWeb, baseUrlApi, gitRepoUrl, gitBranch, databaseType
TestSuite	→ nom + type (UNIT INTEGRATION WEB API UX_WEB UX_MOBILE) + modulePath
TestCase	→ titre + code généré (IA) OU scriptPath (manuel) + testData + flags
Campaign	→ projet + environnement + cas sélectionnés → Run

Flags importants sur un cas de test : - `active` (false → ignoré à l'exécution), - `flaky` (true → réessayé une fois en cas d'échec), - `maxDurationSeconds` (timeout Maven par test), - `springProfile` (injecté en -

`Dspring.profiles.active`), - `databaseType` (résolu : cas de test d'abord, puis environnement), - `generated` (true = code IA, false = script manuel), - `riskLevel` (CRITICAL/HIGH/MEDIUM/LOW — utilisé pour la priorisation d'exécution).

Membres & accès : `ProjectMember` lie un utilisateur à un projet. `ProjectAccessService` vérifie les droits.

3.3 Génération IA de tests — le wizard M1 (ScenarioBuilder)

C'est le cœur « intelligent » pour les tests **UNIT / INTEGRATION**.

Étapes côté utilisateur : 1. **Parcours du code source** via l'API GitHub (`/api/github/.../java-files`) — sans cloner. 2. **Sélection d'une classe** → le frontend en extrait un *squelette* (~15 lignes, méthodes publiques) côté client. 3. **ScenarioBuilder (4 étapes)** : - Choisir la **méthode** à tester (parsée depuis le squelette). - Choisir le **scénario** : `HAPPY_PATH`, `EXCEPTION`, `NULL_INPUT`, `WRONG_INPUT`, `BOUNDARY`. - Le LLM **génère les données de test (JSON)** à partir du schéma Swagger REQUEST → l'humain valide/édite. - Les **assertions** sont auto-construites depuis le schéma Swagger RESPONSE + les données validées. 4. Le payload structuré (`methodName`, `scenarioType`, `expectedBehavior`) + squelette + testData est envoyé à `/api/llm/generate-test`.

Côté LlmService (le prompt engineering) : - Construit un prompt avec un `scenarioBlock` : méthode cible + label du scénario + comportement attendu → force la génération d'**une seule méthode @Test nommée test_{methode}_{scenario}()**. - **Règles UNIT durcies** : `@InjectMocks` sans `new()`, `@BeforeMethod` avec `MockitoAnnotations.openMocks(this)`, mock de *tous* les repositories utilisés, `MockedStatic<SecurityUtils>` si nécessaire, utilisation des valeurs de `testData`. - **Règles de package** : UNIT → `package suites.unit;`, INTEGRATION → `package suites.integration;` (car le fichier est écrit dans `src/test/java/suites/{type}/` du repo cloné). - **Post-traitement** (`postValidate`) : ajoute les imports Mockito manquants, nettoie les blocs markdown (````java` ) et les tokens spéciaux deepseek ( `< | begin_of_sentence | >` ). - **Modèle** : `qwen3-coder-next:cloud` en primaire, **fallback automatique** sur `deepseek-coder:6.7b` si le primaire échoue.

💡 **Point clé pour le jury** : la génération n'est pas « free-form ». Elle est *contrainte* par un scénario structuré, un squelette réel, des données validées par l'humain, et des schémas Swagger. C'est ce qui rend le code généré *compilable et pertinent* plutôt qu'approximatif.

3.4 Exécution des campagnes (ms-execution — 8083)

Pipeline `ExecutionService.runCampaign(campaignId, selectedTestCaseIds)` (asynchrone, `@Async`) :

1. Passe la campagne en `RUNNING`, met à jour `progress` et `currentStep` en continu (suivi temps réel côté UI).
2. **Clone le repo** (`env.gitRepoUrl` @ `env.gitBranch`) — authentification par PAT injecté dans l'URL (`https://oauth2:TOKEN@github.com/...`). Si aucune URL Git → utilise un **template de test IA intégré** (`ai-test-template.zip`).
3. **Priorisation** : les cas sont triés par un *score d'exécution* (`computeExecutionScore`) dérivé du `riskLevel` → les tests critiques passent d'abord.

4. Pour chaque cas (sauf `active=false`) : - Choisit le bon **répertoire de travail** (repo de la suite, ou template intégré pour WEB/API sans repo). - Écrit `testData` dans un fichier temporaire `test-data-{id}.json` (passé en `-Dtest.data.file`). - **Injecte les dépendances** dans le `pom.xml` selon le type (TestNG, Mockito, Selenium + WebDriverManager, REST Assured, Appium, Spring Boot Test, H2/Testcontainers...). - Pour les tests générés : nettoie le code, extrait le nom de classe et le package par regex, écrit le fichier `.java` dans `src/test/java/suites/{type}/`. - Lance `mvnw test -Dtest=...` via `ProcessBuilder`, avec timeout par test (`maxDurationSeconds`) ou global (15 min). - **Parse les rapports Surefire** `target/surefire-reports/TEST-*.xml` pour les résultats méthode par méthode.
5. **Flaky** : un test `flaky=true` qui échoue est rejoué une fois.
6. **Auto-retry M3** (la fonctionnalité phare) : si un test généré échoue avec une **erreur corrigable** (compilation, symbole introuvable, assertion...), `ScriptRetryService` renvoie le code + l'erreur au LLM pour correction, réécrit le fichier, relance Maven — **jusqu'à 3 tentatives**. Tout est tracé dans `retryLog` / `retryCount`.
7. **Analyse IA de l'échec** (`LlmAnalysisService`) : extrait les vraies lignes d'erreur (`[ERROR]`, `cannot find symbol`, `AssertionError`, `NullPointerException`), classe le type d'erreur (COMPILATION_ERROR, ASSERTION_FAILURE, NULL_POINTER, TIMEOUT, RUNTIME_EXCEPTION) et produit une analyse structurée **Cause / Correction / Conseil**.
8. **Capture d'écran** automatique en cas d'échec E2E.
9. Statut final : `FINISHED` ou `FINISHED_WITH_ERRORS` ; **génération PDF automatique**.
10. **Nettoyage** : suppression des répertoires clonés.

Types de tests supportés : UNIT (TestNG+Mockito), INTEGRATION (`@SpringBootTest` +H2/Testcontainers), WEB (Selenium E2E), API (REST Assured), UX/Fonctionnel (agent vision — voir §3.5).

3.5 Intelligence — l'agent autonome d'évaluation UX (le module le plus avancé)

`AgenticEvaluationService` est un **agent IA autonome** qui explore une application web/mobile comme le ferait un testeur humain.

Boucle agentique (`executeEvaluation`) : 1. Lance un **Chrome headless** (Selenium), avec **émulation mobile** possible (iPhone 14, 390×844) ou desktop. 2. **Phase 0 — découverte du menu** : extrait les liens de navigation (texte → URL) pour cartographier l'app. 3. Boucle adaptative (pas de nombre d'étapes fixe ; s'arrête à **saturation**) : - Prend une **capture d'écran** (version HD pour le live view, version compressée pour l'IA). - Envoie l'image au **LLM de vision** (provider commutable : OpenRouter ou Ollama) qui décide de la **prochaine action** (cliquer, remplir, naviguer...). - **Détection de boucle** : si bloqué 4 fois sur la même URL → navigation forcée vers une section non visitée. - **Suivi de couverture** : URLs visitées, sections du menu couvertes, éléments interagis. - **Test fonctionnel** : `WebFunctionalTester` détecte les formulaires de la page et les teste automatiquement (saisie + soumission + vérification). - **Human-in-the-loop (HITL)** : le testeur peut **mettre en pause / reprendre / arrêter** l'agent en temps réel ; un mode `SUPERVISED` demande validation. - **Streaming live** (`EvaluationStreamService`) : les étapes sont poussées en direct vers le frontend (SSE). 4. **Arrêt par saturation** : quand l'agent ne découvre plus rien de nouveau pendant N étapes (seuil configurable), il considère le tour complet et s'arrête — un plafond de sécurité (`max-steps`, 60) évite les boucles infinies. 5. **Mobile natif (APK)** : pour `MOBILE_APP`, l'agent pilote un vrai device/émulateur via **Appium** (`AppiumDriverService`).

💡 **L'argument différenciant** : ce n'est pas un script de test enregistré. L'agent *décide* de ses actions à partir de ce qu'il voit, s'adapte à n'importe quelle application, détecte sa propre saturation, et accepte le contrôle humain en cours de route.

3.6 Sécurité — scans réels (ms-execution + ms_gestion)

Trois types de scans, **exécutés réellement** via des outils standards :

Type	Outil	Cible	Ce qu'il trouve
SAST	Semgrep (<code>semgrep scan --config auto --json</code>)	Code source cloné	Faibles dans le code (injection SQL, XSS, secrets en dur...)
DAST	OWASP ZAP	URL de l'app en cours d'exécution (<code>baseUrlApi</code> / <code>baseUrlWeb</code>)	Faibles à l'exécution (headers, endpoints exposés...)
SCA	OWASP Dependency-Check	Dépendances du projet	CVE connues dans les librairies tierces

Traitement des résultats (`SecurityScanService` , asynchrone) : - Chaque scan crée un `SecurityScan` (statut RUNNING → COMPLETED/FAILED) avec `scanRef` , moteur, branche, commit, durée, fichiers/lignes analysés. - Chaque finding devient une `SecurityVulnerability` avec : sévérité (CRITICAL→INFO), `cweId` , **mapping CWE** → **OWASP Top 10** (ex. CWE-89 → A03 Injection), fichier:ligne ou endpoint, snippet, recommandation, risque. - Pour le SAST, calcul d'une **densité de défauts** → score de couverture.

Côté gestion (lecture) : `SecurityService` (ms_gestion) fournit le **dashboard** : score de sécurité global, sous-scores par catégorie (OWASP, secure coding, infra, dépendances), répartition par sévérité/statut, **grille OWASP Top 10** (pass/warn/fail par catégorie), conformité.

Cycle de vie d'une vulnérabilité : statut `OPEN` → `IN_PROGRESS` → `RESOLVED/CLOSED/FALSE_POSITIVE` . Assignable à un membre (notification + email). **Quand elle passe en RESOLVED, le propriétaire du projet est notifié** (fonctionnalité que je viens d'ajouter).

3.7 Rapports PDF (ReportService)

Généré automatiquement en fin de campagne : - **Page de garde** : projet, version, environnement, badge de statut. - **Résumé exécutif** : table KPI (total/succès/échecs/erreurs/durée) + jauge de progression. - **Contexte** : projet, suites, cas, environnement (repo, branche, type de BD). - **Résultats détaillés** : carte par test avec type d'erreur classifié. - **Analyse IA** : lignes d'erreur extraites + analyse structurée. - **Métriques** : top 5 des tests les plus lents, durée moyenne par type. - **Tests flaky, recommandations IA complètes, annexe** des scripts générés. - Les rapports utilisent des endpoints `/api/internal/` (sans contrôle utilisateur) pour résoudre les vrais noms projet/campagne lors de la génération automatique (pas de contexte utilisateur).

3.8 Collaboration : assignments & notifications

- **Notifications** in-app (cloche) : types `VULN_ASSIGNED`, `VULN_RESOLVED`, `TEST_FAILED`, `TEST_RESOLVED`, `CAMPAIGN_COMPLETED`, `REPORT_READY`, `SCAN_COMPLETED`. Lien cliquable → redirige vers la bonne page.
- **Page « Mes Assignements »** : regroupe toutes les vulnérabilités et erreurs de test assignées à l'utilisateur, en 2 onglets. Cartes avec sévérité, projet, assigné à. Bouton « **Résoudre** » (tick vert) → notifie le propriétaire du projet.
- **Emails** : envoi d'un email à l'assigné (si SMTP configuré — fonctionne sans, dégradation gracieuse).

4. Scénario de démo (chronométré et reproductible)

Objectif : ~12-15 min de démo fluide qui montre la *boucle complète* + les 3 différenciateurs (auto-correction IA, agent UX vision, sécurité réelle).

Préparation (avant le jury — voir checklist §8)

Tout doit déjà tourner : Eureka, Auth, ms_gestion, ms-execution, Frontend, PostgreSQL, Ollama. Avoir **un projet de démo déjà créé** avec un environnement pointant sur un repo GitHub de test (avec `mvnw`).

Déroulé

[0:00 — 1:30] Introduction & connexion - Pitch (§1.3) en une phrase. - Connexion → arrivée sur le **Dashboard** (KPIs globaux : projets, campagnes, taux de réussite). - « L'interface est organisée par le cycle de vie du test : Projets → Campagnes → Résultats → Intelligence → Sécurité. »

[1:30 — 3:30] Le projet et sa configuration - Ouvrir le projet de démo → montrer **Environnements** (repo Git, branche, type de BD, URLs). - Montrer les **Suites** (UNIT, INTEGRATION, WEB) et expliquer : « toute la config technique vit sur l'environnement, ce qui permet de rejouer les mêmes tests sur dev/staging/prod ». - Montrer **Membres & rôles**.

[3:30 — 6:30] Génération IA d'un test unitaire (le wizard M1) — MOMENT FORT - Créer un cas de test → lancer le **ScenarioBuilder**. - Parcourir le code source via GitHub, choisir une classe, une **méthode**, un **scénario** (ex. `HAPPY_PATH`). - Laisser l'IA **générer les données de test** depuis le schéma Swagger → valider. - Générer le test → montrer le **code TestNG+Mockito produit** (commenter : `@InjectMocks`, mocks, assertions). - « Le point clé : ce code va être réellement compilé et exécuté, pas juste affiché. »

[6:30 — 9:30] Exécution + Auto-correction IA (M3) — MOMENT FORT - Créer une **campagne** avec ce test (+ d'autres déjà prêts), choisir l'environnement, **Run**. - Montrer la **progression temps réel** (progress %, currentStep). - Si possible, avoir un cas qui échoue d'abord puis se corrige : montrer dans les logs/ `retryLog` que **l'IA a corrigé le script et relancé** (« AUTO-RETRY 1/3 ... SUCCÈS après correction »). - Aller dans

Résultats : statut par test, **analyse IA structurée** (Cause/Correction/Conseil). - Ouvrir le **rapport PDF** généré automatiquement.

[9:30 — 12:00] Agent UX autonome (Intelligence) — MOMENT FORT - Aller dans **Intelligence** → créer une évaluation fonctionnelle sur l'URL de l'app. - Lancer → montrer le **live view** : l'agent navigue, le menu détecté, les étapes en streaming, les **formulaires testés automatiquement**. - Montrer le **HITL** : mettre en **pause**, puis reprendre. - Commenter : « L'agent voit les captures via un LLM de vision et décide seul de la prochaine action. Il s'arrête quand il a tout couvert (saturation). »

[12:00 — 14:00] Sécurité réelle - Aller dans **Security** → lancer un scan **SAST** (Semgrep) sur le projet. - Montrer le **dashboard** : score de sécurité, **grille OWASP Top 10**, liste des vulnérabilités avec sévérité, CWE, fichier:ligne. - Ouvrir une vulnérabilité → **l'assigner** à un membre → montrer la **notification** générée. - Aller dans **Mes Assignements** → cliquer **Résoudre** (tick vert) → « le propriétaire du projet est notifié ».

[14:00 — 15:00] Conclusion - Récapituler la **boucle fermée** : code → génération → exécution → auto-correction → analyse → rapport, + couches UX et sécurité. - Ouvrir sur les perspectives (§ « What's Left To Do » du projet).

Plan B (si une démo live échoue)

- Avoir des **captures d'écran / un PDF de rapport déjà généré** et une **campagne déjà exécutée** sous la main.
 - Pour l'agent UX : avoir une **évaluation déjà terminée** avec ses étapes enregistrées à montrer.
 - Ne jamais dépendre uniquement du live : le LLM local et les outils externes peuvent être lents.
-

5. Justification des décisions techniques (le « pourquoi »)

Le jury demande toujours « pourquoi ce choix et pas un autre ? ». Réponses préparées :

Décision	Pourquoi	Alternative écartée
Microservices	Séparation claire gestion / exécution ; l'exécution est lourde (clone+Maven) et doit pouvoir scaler/isoler indépendamment	Monolithe : plus simple mais couplage fort, pas d'isolation des charges lourdes
LLM local (Ollama)	Le code source du client ne sort jamais de l'infra → confidentialité ; coût nul ; pas de dépendance à un fournisseur	API cloud (OpenAI...) : meilleure qualité mais fuite de code + coût + dépendance
TestNG (pas JUnit)	Meilleure gestion des groupes, du parallélisme et des <code>@BeforeMethod</code> / <code>@AfterMethod</code> pour les patterns Mockito ; provider Surefire configuré	JUnit 5 : équivalent, choix d'homogénéité
Base partagée	Cohérence immédiate, simplicité pour un PFE, pas de duplication de données projet	Database-per-service : plus « pur » mais nécessite synchronisation/événements (overkill ici)
Génération contrainte (ScenarioBuilder)	Un LLM 6.7B sur 4 Go de VRAM produit du code approximatif en free-form → on le <i>guide</i> (méthode, scénario, squelette, données validées)	Prompt libre : code souvent non compilable
Auto-retry IA en boucle fermée	Un test généré qui ne compile pas est inutile → l'IA se corrige seule, zéro intervention	Échec définitif + correction manuelle
Vision LLM pour l'UX	Pour tester l'UX d' <i>n'importe quelle</i> app sans script pré-écrit, l'agent doit « voir » l'écran	Scripts Selenium figés : cassent au moindre changement d'UI
Auth JWT en cookies	Stateless, pas de session serveur, cookies HttpOnly = protégés du JS	Session serveur : état à gérer, scaling plus dur
API Routes Next.js comme proxy	Le front ne parle jamais directement au backend → on cache les URLs internes et on centralise le relais JWT	Appels directs front→backend : CORS + exposition des URLs internes

6. Préparation aux questions du jury

Organisé par thème. Pour chaque question : la **réponse courte** à donner.

6.1 Architecture

Q : Pourquoi des microservices et pas un monolithe ? R : Parce que l'exécution des tests est une charge lourde et isolée (clone Git, processus Maven, navigateurs headless, scans sécurité). La séparer de la gestion permet de l'isoler, de la scaler indépendamment, et d'éviter qu'un test qui plante n'affecte l'API de gestion. Eureka assure la découverte des services sans URLs en dur.

Q : Comment les services communiquent-ils ? R : Eureka pour la découverte. Le Frontend passe par ses API Routes qui relaient le JWT. Entre services, appels REST ; ms-execution lit la base partagée via des entités miroirs et utilise des endpoints `/api/internal/` de ms_gestion pour les données sans contexte utilisateur.

Q : Pourquoi une base partagée et pas une base par service ? R : Choix pragmatique pour le périmètre du PFE — cohérence transactionnelle immédiate et pas de duplication des données projet. Je connais le pattern database-per-service ; le passage se ferait en introduisant des événements (ex. Kafka) pour synchroniser, ce que je documente comme évolution.

6.2 IA / LLM

Q : Quel modèle, et pourquoi local ? R : `qwen3-coder-next` en primaire, `deepseek-coder:6.7b` en fallback, via Ollama. Local pour la **confidentialité** (le code source ne sort jamais), le coût nul et l'indépendance fournisseur. Contrainte : GPU 4 Go → je guide fortement le prompt.

Q : Comment garantir que le code généré est correct ? R : Trois garde-fous. (1) **Génération contrainte** : scénario structuré + squelette réel + données validées par l'humain + schémas Swagger. (2) **Post-validation** : ajout des imports manquants, nettoyage des tokens. (3) **Exécution réelle + auto-retry** : si ça ne compile pas, l'IA corrige jusqu'à 3 fois. Au final, seul un test qui *passé réellement* est validé.

Q : Que se passe-t-il si l'IA n'arrive pas à corriger ? R : Après 3 tentatives, le test est marqué en échec avec son `retryLog` complet et une analyse IA Cause/Correction/Conseil, et l'humain reprend la main. La traçabilité est totale.

Q : L'agent UX, c'est juste du Selenium scripté ? R : Non. C'est un agent autonome : il prend des captures, les envoie à un **LLM de vision** qui décide de l'action suivante. Il découvre le menu, détecte ses boucles, suit sa couverture, s'arrête à saturation, teste les formulaires, et accepte le contrôle humain (pause/reprise). Selenium n'est que le *bras* qui exécute ; le *cerveau* est le LLM.

Q : Hallucinations du LLM ? R : Limitées par la contrainte du prompt et surtout par la **boucle de réalité** : un test halluciné ne compile pas → rejeté/corrigé. Pour l'analyse d'échec, on n'envoie au LLM que les *vraies* lignes d'erreur extraites, pas du texte inventé.

6.3 Sécurité (du produit lui-même)

Q : Comment sont protégées les API ? R : Spring Security, `.anyRequest().authenticated()`, JWT signé (HS512), rôles vérifiés par `@PreAuthorize`. Seuls les endpoints internes service-to-service sont en `permitAll`, et ils ne portent pas de données sensibles utilisateur.

Q : Et les tokens GitHub / mots de passe ? R : (réponse honnête — voir §7) Aujourd'hui le PAT GitHub et les identifiants BD sont dans `application.properties` pour le développement. En production, je les externalise en **variables d'environnement / secret manager** (Vault, ou secrets Docker/K8s) — le code lit déjà certaines valeurs via `${VAR:défaut}`.

Q : Les scans de sécurité sont-ils réels ou simulés ? R : Réels. On exécute Semgrep (SAST), OWASP ZAP (DAST) et OWASP Dependency-Check (SCA) en tant que vrais processus, on parse leur sortie JSON, et on mappe les CWE vers l'OWASP Top 10.

6.4 Exécution / fiabilité

Q : Comment isolez-vous les exécutions ? R : Chaque campagne clone le repo dans un répertoire temporaire dédié, exécute Maven dans ce répertoire, puis le supprime. Timeout par test et global. Les tests flaky sont rejoués.

Q : Tests d'intégration avec base de données ? R : `@SpringBootTest` + H2 (en mémoire) ou Testcontainers (PostgreSQL/MySQL/MongoDB réels dans des conteneurs jetables). Les dépendances et propriétés sont injectées dans le pom et l'application-test au moment de l'exécution.

Q : Que se passe-t-il si le repo n'a pas de Maven wrapper ? R : (*honnête*) Actuellement on cherche `mvnw` / `mvnw.cmd` ; le fallback vers un `mvn` système est une amélioration identifiée. Pour la démo, les repos cibles ont leur wrapper.

6.5 Frontend / UX

Q : Pourquoi Next.js et le proxy par API Routes ? R : Le front ne parle jamais directement aux microservices : les API Routes cachent les URLs internes, gèrent CORS et **centralisent le relais du JWT** (cookie → header Bearer). C'est aussi ce qui m'a permis de corriger proprement un bug d'authentification récent.

6.6 Questions « méta » / gestion de projet

Q : Qu'est-ce qui a été le plus difficile ? R : Fiabiliser la génération IA sur un petit modèle local. La solution : passer du prompt libre à une génération *contrainte et structurée* + une boucle d'auto-correction qui exécute réellement le code.

Q : Si c'était à refaire ? R : J'introduirais plus tôt les migrations SQL versionnées (Flyway) et l'externalisation des secrets, et je découperais peut-être la sécurité en service dédié.

7. Limites connues & réponses honnêtes

Le jury *cherche* les faiblesses. Les connaître et les assumer = crédibilité maximale.

1. **Secrets en clair dans `application.properties`** (PAT GitHub `github.token`, mot de passe BD, `jwt.secret`). → *Réponse* : acceptable en dev, à externaliser en variables d'environnement / secret manager en prod. Le code supporte déjà `${VAR:défaut}`. **Avant la soutenance, idéalement régénérer le PAT GitHub** (il est committé, donc compromis).
2. **Base partagée (pas database-per-service)**. → *Réponse* : choix pragmatique assumé ; évolution documentée via événements.

3. **ddl-auto=update sur ms_gestion**. → *Réponse* : pratique en dev ; en prod, passer à **validate** + migrations Flyway/Liquibase.
 4. **Modèle LLM contraint (GPU 4 Go)**. → *Réponse* : d'où la génération guidée + l'auto-retry. Un modèle plus gros améliorerait la qualité du premier jet.
 5. **Couverture de tests de la plateforme elle-même** (méta-tests) limitée. → *Réponse* : le focus a été la chaîne fonctionnelle ; ajouter des tests unitaires sur les services backend est la prochaine étape.
 6. **Maven wrapper requis** (pas de fallback **mvn** système). → *Réponse* : amélioration identifiée.
 7. **Dépendance à des outils externes** (Semgrep, ZAP, Dependency-Check, Ollama, Chrome) qui doivent être installés sur la machine d'exécution. → *Réponse* : à conteneuriser (Docker) pour la reproductibilité — déjà partiellement en place.
 8. **SMTP email** échoue si **MAIL_PASSWORD** vide. → *Réponse* : dégradation gracieuse — l'assignement et la notification in-app fonctionnent sans email.
-

8. Checklist pré-démo (à faire la veille et le matin)

Infrastructure (dans l'ordre de démarrage) : - [] PostgreSQL démarré (**docker-compose up -d** dans **infrastructure/**) — BD **Test_platform_db** accessible. - [] **Eureka** (8761) démarré en premier. - [] **Auth** (8081) démarré. - [] **ms_gestion** (8082) démarré. - [] **ms-execution** (8083) démarré. - [] **Frontend** (**npm run dev** , 3000). - [] **Ollama** (11434) lancé + modèles **qwen3-coder-next** et **deepseek-coder:6.7b** déjà téléchargés (**ollama list**).

Outils CLI dans le PATH : - [] **semgrep --version** OK. - [] OWASP ZAP installé/accessible. - [] Dependency-Check installé. - [] **mvn / mvnw** + JDK 17 OK. - [] Chrome installé (pour l'agent UX).

Données de démo prêtes : - [] Un **utilisateur admin** + un utilisateur « membre » pour montrer l'assignement. - [] Un **projet de démo** avec un **environnement** pointant sur un repo GitHub de test (avec **mvnw**). - [] Au moins **une suite UNIT** et **une campagne déjà exécutée avec succès** (Plan B). - [] **Un rapport PDF déjà généré** sous la main. - [] **Une évaluation UX déjà terminée** (Plan B si le live est lent). - [] Le **PAT GitHub valide** (et idéalement régénéré).

Vérifications finales : - [] Connexion OK, Dashboard charge. - [] Page « Mes Assignements » charge (vérifie le relais JWT). - [] Cloche de notifications fonctionne. - [] Connexion réseau stable (l'agent vision via OpenRouter en a besoin, sauf si Ollama vision local).

Annexe — Glossaire rapide pour répondre vite

- **SAST** : analyse statique du code source (sans l'exécuter).
- **DAST** : analyse dynamique de l'app en cours d'exécution.

- **SCA** : analyse des dépendances (CVE).
- **CWE** : catalogue standard des types de faiblesses (ex. CWE-89 = injection SQL).
- **OWASP Top 10** : classement des 10 risques de sécurité web majeurs (A01...A10).
- **LLM agentique** : un LLM qui *agit* en boucle (observer → décider → agir), pas un simple générateur de texte.
- **Vision LLM** : modèle multimodal qui prend une image en entrée.
- **HITL (Human-in-the-loop)** : l'humain peut intervenir pendant l'exécution automatique.
- **Surefire** : plugin Maven qui exécute les tests et produit les rapports XML.
- **Flaky test** : test instable qui réussit/échoue de façon non déterministe.
- **Saturation (agent)** : état où l'agent ne découvre plus rien de nouveau → fin d'exploration. ``